

SUMMERSHIP 2026 · INTERNSHIP PROJECT BRIEF

AjraVerify — a deterministic safety gate for AI-generated agricultural answers

Ajrasakha · github.com/vicharanashala/ajrasakha · Feature branch `feature/agricultural-answer-verification` · Module `backend/src/modules/verification/` · Prepared 2026-10-02

AI answers to farmers' questions used to ship with no checking of their factual claims. AjraVerify now inspects every AI-generated answer after it is produced and before it is delivered, checks its claims against the product's own trusted catalogues, returns one of four verdicts, and writes an audit receipt for every decision. Unsafe or unverifiable claims escalate into the expert review workflow that already existed. Milestone 1 uses **zero LLM calls** — it is regex extraction plus catalogue lookups, so it is offline-testable, free per answer, and cannot hallucinate a restriction.

36

offline tests passing

4

verdicts per answer

0LLM calls on the
decision path**18**

audit receipts recorded

2

pipeline seams wired

1 · The problem this solves

Ajrasakha answers farmers' agricultural questions, prioritising verified expert knowledge but falling back to AI-generated answers. Those AI answers carry dosage and timing instructions — exactly the claims where a mistake costs a farmer money, crop or health. Before this work the only check was a human reading the answer later.

A realistic AI answer the pipeline used to ship unchecked:

"Spray monocrotophos 36% SL at 25 ml per litre of water, about 500 ml per acre.
Repeat the spray every 5 days for 3 sprays. Spray during early morning for best results."

If monocrotophos is marked **Restricted** in the product's own catalogue, this answer must not reach a farmer unreviewed. Nothing in the pipeline knew.

2 · What was built

1 · Extract

Regex recognises claim *shapes*: dosages (ml/L, g/L, ml/acre, kg/acre, %, ppm), intervals → days, spray counts, chemical-name candidates. No agricultural facts in code.



2 · Resolve

Look names up in `chemical_master` then `crop_master` (type chemical, aliases included). Statuses come from the documents.



3 · Decide

Deterministic counters → one of four verdicts with a human-readable reason. No LLM, no randomness.



4 · RecordAjraVerify · Ajrasakha · github.com/vicharanashala/ajrasakha · Summership 2026 submission brief

A receipt document is persisted for every decision: claims, per-claim resolutions, reason, counters, policy thresholds, timestamp.

**5 · Escalate**

Verdict + receipt ride on the question into the existing PAE-expert / moderator review workflow.

The four verdicts

Verdict	Meaning	What happens
VERIFIED	Every factual claim matched trusted data within tolerance.	Answer ships with its receipt.
CONDITIONAL	All claims matched, but some values sit in the $\pm 10\%$ caution band.	Answer ships <i>with the receipt attached</i> for the record.
REVIEW_REQUIRED	A claim — or a dose that cannot be attributed to any product — matched nothing trusted.	Escalates into the existing expert review workflow.
BLOCKED	A restricted/banned chemical is named, or a value is beyond the 50% safety tolerance.	Must not ship without expert approval.

Precedence: BLOCKED > REVIEW_REQUIRED > CONDITIONAL > VERIFIED. Policy thresholds live in one object: near-miss $\pm 10\%$, blocked 50%, restricted statuses ['restricted', 'banned'].

Module map — backend/src/modules/verification/

File	Responsibility
extractors/RuleBasedClaimExtractor.ts	Regex claim extraction: dosage units, interval ranges normalised to days, spray counts (digits/words/ordinals), chemical-name candidates across four passes (before-dose run, "name NN% formulation", product shapes like 2,4-D, "of/with" and interleaved shapes).
services/ClaimResolver.ts	Catalogue lookups; Restricted/Banned read from the document; guards against empty-subject lookups (an empty regex <code>\b\b</code> would match any document).
services/PolicyEngine.ts	Deterministic counters → verdict + reason. Reads structured fields only (status, valueVerdict, restrictedChemical), never its own prose.
services/VerificationService.ts	Orchestrator: extract → resolve → decide → persist. A receipt write failure can never change the decision.
repositories/VerificationReceiptRepository.ts	verification_receipts collection with indexes on questionId, answerId (sparse), createdAt, verdict + createdAt.
controllers/VerificationController.ts	POST /api/verification/check, GET /api/verification/receipt/:questionId, GET /api/verification/receipts?limit&verdict — existing auth middleware + DTO validation.
classes/validators/, container.ts, index.ts, types/	DTOs, DI wiring and domain types. Module is auto-discovered by the repo's folder convention — no manual registration.
tests/ (4 files)	36 offline specs: extractor shapes, all four verdicts and tolerance edges, catalogue guards, verdict routing, persistence-failure isolation.

Where it hooks into the pipeline

AjraVerify · Ajrasakha · github.com/vicharanashala/ajrasakha · Summership 2026 submission brief

Seam	Behaviour	Safety property
Ingestion workers/questionProcessor.worker.ts	After the AI initial answer is generated or supplied in a bulk upload, the gate runs and stores aiAnswerVerification {verdict, reason, receiptId} on the question. The question id is pre-allocated so the receipt is linked to the question it judged.	Verification failures are logged and swallowed — ingestion can never stall because of the gate.
Shipping modules/answer/services/AnswerApprovalService.ts	The WhatsApp / AJRASAKHA AI-ship path verifies the answer text before it is marked shipped and stores the verdict on the question.	Same non-blocking rule. Enforcement (withholding a BLOCKED answer) is left as a maintainer policy decision, with the seam already in place.

Both seams run behind the configuration flag ENABLE_AJRAVERIFY — off by default, so nothing changes for existing deployments until it is switched on.

3 · Evidence: before and after, same answer text

	Before	After
Ingested question	6abe12e0997a10320b06553d	6abf84086c4139d68ea7f5bc
Verification in the pipeline	none	gate runs after the AI answer, before delivery
Verdict stored on the question	aiAnswerVerification: null	{verdict: "REVIEW_REQUIRED", reason: "1 claim could not be matched against trusted data – expert review required before delivery.", receiptId: "6abf84086c4139d68ea7f5bd"}
Audit receipt	none for this question	GET /api/verification/receipt/6abf84086c4139d68ea7f5bc → 200, linked, dosage claims attributed to subject monocrotophos

Both questions carry the same unsafe AI answer shown on page 1. The "before" question is kept in the local database on purpose as the unverified artefact.

4 · Evidence: the verdict follows the data, not the code

The identical answer text was checked three times against the running system. The **only** difference was one catalogue document — no code, no config, no re-deploy.

Catalogue state for Monocrotophos	Verdict	Reason recorded in the receipt	Receipt
no entry	REVIEW_REQUIRED	"1 claim could not be matched against trusted data — expert review required before delivery."	6abf81be...5ac8
entry, status: "Restricted"	BLOCKED	"Blocked: 1 restricted/banned chemical in the trusted catalogue. This answer must not ship without expert approval."	6abf81cf...5aca
entry, status: "Approved"	VERIFIED	"All factual claims matched trusted data within tolerance."	6abf81cf...5acb

A dose with no product named — "Apply about 500 ml per acre of water." — is checked against nothing, so it escalates: REVIEW_REQUIRED , receipt 6abf81be...5ac9. Catalogue seeds were removed after the demonstration; all receipts remain as the audit trail.

Receipt store at the time of writing

VERIFIED 5 · CONDITIONAL 0 · REVIEW_REQUIRED 10 · BLOCKED 3 · total 18

Filtered queries return a total that matches the filter. Verified live: blank body → 400, unknown verdict → 400, malformed receipt id → 400, unknown question → 404, and both authentication modes (internal API key and Firebase JWT) accepted.

5 · Evidence: what running it for real caught

Unit tests passed while four real defects were still present. Running the gate against the database — with the text the AI actually produces — exposed them. All four are fixed, each with a regression test (test count 25 → 36).

#	Defect	Why it mattered	Fix
	AjraVerify · Ajrasakha · github.com/vicharanashala/ajrasakha · Summership 2026 submission brief		
1	The chemical name was lost behind a formulation code. "monocrotophos 36% SL at 25 ml per litre" attributed the doses to "sl at" and "about".	With a restricted chemical in the catalogue this resolved "not found" → review, instead of BLOCKED — a false go-ahead in the dangerous direction.	Filler / formulation-token rejection, a new extraction pass for "name NN% formulation", and sentence-scoped positional attribution of doses to the nearest mention.
2	A dose with no product passed as verified. "Apply about 500 ml per acre of water" resolved as "matched" although nothing had been compared.	The gate would certify a dose it never actually checked.	An unattributable dose now resolves "not found" → REVIEW_REQUIRED .
3	Ingestion receipts were unlinked. questionId was null on 12 of 17 receipts.	The per-question audit lookup 404'd for every ingested question — the audit trail could not be followed.	The question id is pre-allocated and passed to the gate; verified by a 200 with the linked id.
4	Filtered totals ignored the filter — every verdict query reported the collection size.	Any dashboard built on this API would show impossible numbers.	Counts now honour the filter; the list and its total always agree.

Lesson worth keeping

Unit tests validate the shapes you imagine. Only real answer text against real data shows you which claims you silently dropped. That round turned "green tests" into "verified behaviour" — and it is the most useful thing I learned in this project.

6 · Quality

- **36 offline tests, 4 files, all passing** — extractor 16, policy 8, resolver 4, service 8. No database, no Firebase, no API keys, no network: the suite runs anywhere CI runs.
- **tsc --noEmit clean** with the module included in the production build (test files typecheck too — this repo's tsconfig only excludes *.spec.ts).
- **Full backend suite unchanged:** the repository has 21 pre-existing failures in auth / crop / question tests — test drift verified as pre-existing by stashing only my tracked changes and re-running. Untouched by this work.
- **Failure isolation:** a receipt write failure is logged and cannot alter the verdict; a gate failure in ingestion cannot stall ingestion.

7 · Honest limitations (Milestone 1 scope)

Limitation	Why it matters, and what unblocks it
Catalogues carry no per-chemical dose limits yet.	Dosage <i>values</i> are recorded as expert evidence rather than compared; name and restricted/banned status are compared today. A gate is only as strict as its data — adding dose limits unlocks the $\pm 10\%$ / $\pm 50\%$ bands.
Interval and spray-count claims are recorded as unresolved .	No trusted schedule source is wired yet; the claims are captured for the expert instead of being dropped.
Extraction is English / Latin-script regex .	Paraphrase, mixed-language and regional-script answers are Milestone 2 (an LLM extractor behind the same interface; the deterministic policy engine remains the only decision-maker).
Enforcement is evidence-only .	The gate flags and records; it does not unilaterally withhold a BLOCKED answer. That is a maintainer policy decision, and the seam is ready for it.
No dashboard tab and no frontend surface beyond the API.	The receipts API is dashboard-ready (verdict filters, totals, per-question lookup).

Next milestones, in order

1. Per-chemical dose limits in the catalogue → unlocks real dosage comparison.
2. Interval resolution against Package-of-Practices documents.
3. Enforced withholding of BLOCKED answers.
4. Milestone 2: LLM-assisted extraction behind the same interface, config-switched.
5. Admin receipts dashboard tab.

8 · Two companion fixes (small, separate PRs)

Fix	Problem and resolution
MongoDB TLS shared/database/providers/mongo/MongoDatabase.ts	The driver hard-coded <code>ssl: true, tls: true</code> on every client, breaking any non-Atlas <code>mongodb://</code> server with <code>SSLHandshakeFailed</code> . TLS is now enabled only for <code>mongodb+srv://</code> URIs or URIs carrying <code>ssl/tls=true</code> . Verified against a local mongod.
Environment typo frontend/src/shared/app.ts	Read the misspelled key <code>VITE_IS_DEVELPOMENT</code> while the example env file already used <code>VITE_IS_DEVELOPMENT</code> , so the development flag never matched. Fixed to the correct key.

9 · How to see it working (five minutes)

```
cd backend && pnpm exec vitest run src/modules/verification      # 36 offline tests

# 1. Gate one answer (admin/expert JWT or internal API key)
curl -s -X POST localhost:3141/api/verification/check \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"answerText":"Spray monocrotophos 36% SL at 25 ml per litre of water, about 500 ml per acre. Repeat every 5 days for 3 sprays."}'
```

2. The audit trail, then the verdict distribution

```
curl -s "localhost:3141/api/verification/receipts?limit=5" -H "Authorization: Bearer $TOKEN"
curl -s "localhost:3141/api/verification/receipts?verdict=BLOCKED" -H "Authorization: Bearer $TOKEN"
```

3. The receipt attached to an ingested question

```
curl -s "localhost:3141/api/verification/receipt/6abf84086c4139d68ea7f5bc" -H "Authorization: Bearer $TOKEN"
```

To reproduce the verdict flip, insert a `chemical_master` document named `Monocrotophos` with `status: "Restricted"`, re-run step 1 → **BLOCKED**; change the status to `"Approved"` and re-run → **VERIFIED**. No code change either time. Remove the document afterwards.

10 • Submission artefacts

Artefact	Where
Feature code	backend/src/modules/verification/ (new module) · small edits to workers/questionProcessor.worker.ts, modules/answer/services/AnswerApprovalService.ts, config/app.ts, types.ts
Companion fixes	shared/database/providers/mongo/MongoDatabase.ts · frontend/src/shared/app.ts
Before / after report	docs/ajraverify-before-after.md (+ .html visual) · docs/ajraverify-submission-brief.pdf (this document)
Dated build log	docs/ajraverify-buildlog.md (2026-10-01 M1, 2026-10-02 M1.5 + defect round)
Context for review	docs/ajraverify-submission-context.md · docs/ajraverify-plan.md
Evidence receipts	6abf81be...5ac8 · 6abf81be...5ac9 · 6abf81cf...5aca · 6abf81cf...5acb · 6abf84086c4139d68ea7f5bd
Before / after questions	6abe12e0997a10320b06553d (no verdict) · 6abf84086c4139d68ea7f5bc (verdict + linked receipt)

11 • Anticipated questions

"Why not just ask the LLM to check its own answer?"

An LLM cannot reliably state what the catalogue says and cannot prove a restriction. The gate resolves against documents and stores the evidence; in Milestone 2 the LLM only *finds* candidate claims and never decides.

"Is regex extraction reliable?"

Not alone — which is why the design favours recall: an unrecognised chemical escalates to a human instead of passing. Precision comes from the catalogue lookup, and every receipt shows exactly what matched.

"What if the catalogue is empty or wrong?"

Then the gate escalates rather than waving answers through: an empty catalogue produced **REVIEW_REQUIRED** for the demo answer. Data quality becomes a visible verdict, not a silent gap.

"Can this block a good answer?"

In Milestone 1.5 it is evidence-only by design; enforcement is a maintainer policy decision. A wrong catalogue entry causes visible escalation to an expert, never silent harm.

"What is the biggest limitation?"

No per-chemical dose limits in the catalogue yet, so dosage values are recorded for expert review instead of compared. Adding that data unlocks the tolerance bands — it is the first roadmap item.

Prepared 2026-10-02 · Ajrasakha · AjraVerify (feature branch `feature/agricultural-answer-verification`) · All figures in this brief come from the running local system: receipts, question documents, test output and HTTP responses.